School-Admin Side Features - Mobile App Handoff

Audience: developers building the SmartTrack mobile app (driver, guest driver, and parent roles). **Purpose:** the mobile app shares one backend with the school-admin/super-admin web console. This document describes what the admin side manages, what data and endpoints already exist for the mobile roles to consume, and - most importantly - what's *missing* on the backend that mobile development will need before certain features can work end-to-end.

Base API URL: `http://<host>/api`. Every route except `/api/auth/login`, `/forgot-password`, `/verify-otp`, `/reset-password` and `/health` requires `Authorization: Bearer <JWT>`.

---

1. Roles and who uses what

Role	Surface
`super_admin`	Web console - manages schools, plans, subscriptions, platform-wide reports
`school_admin`	Web console - manages one school's students, buses, drivers, routes, attendance, leave, notifications, support
`driver`	**Mobile app** - drives assigned trips, reports live GPS, marks attendance
`guest_driver`	**Mobile app** - same as driver, for one-off substitute trips created via the Guest Drivers flow
`parent`	**Mobile app** - views their child's bus/attendance/leave, receives notifications

All four roles authenticate through the same `POST /api/auth/login`, returning `{ user, token }`. The JWT payload is minimal - `{ id, role, school_id }` - so any per-request detail (name, email, fcm_token) comes from the `user` object at login time or `GET /api/auth/me`, not from the token itself.

---

2. Data model (shared between web and mobile)

These are the exact TypeScript shapes already used by the web app (`frontend/src/types/index.ts`) - reuse them as-is in the mobile app rather than inventing parallel types.

```
type UserRole = 'super_admin' | 'school_admin' | 'driver' | 'guest_driver' | 'parent'
type AttendanceStatus = 'present' | 'absent' | 'leave'
type LeaveStatus = 'pending' | 'approved' | 'rejected'
type TransferStatus = 'initiated' | 'in_progress' | 'completed'
type GuestTripStatus = 'pending_approval' | 'approved' | 'rejected' | 'completed'
type TripType = 'pickup' | 'drop'
type TripStatus = 'not_started' | 'in_progress' | 'completed'

interface User {
 id: string; name: string; email: string; phone?: string; role: UserRole
 school_id?: string; school_name?: string; avatar?: string
```

```
 fcm_token?: string; created_at?: string; last_login?: string
}

interface ParentDetail {
 id: string; student_id: string; parent_name: string; relationship: string
 email: string; phone: string; whatsapp: string
}

interface Student {
 id: string; school_id: string; name: string; class: string; division: string
 roll_number: string; dob: string; photo_url?: string; student_qr_code?: string
 is_active: boolean; pickup_stop_id?: string; drop_stop_id?: string
 route_name?: string; parents: ParentDetail[]; created_at: string
}

interface Bus {
 id: string; school_id: string; bus_number: string; seat_capacity: number
 make_model?: string; year?: number; insurance_expiry?: string; fitness_cert_expiry?: string
 safety_qr_code?: string; is_active: boolean; current_trip_id?: string
 driver_id?: string; driver_name?: string
 status?: 'running' | 'idle' | 'offline'; current_stop?: string; created_at: string
}

interface Stop {
 id: string; route_id: string; name: string; latitude: number; longitude: number
 order_index: number; estimated_time?: string; student_count?: number
}

interface Route {
 id: string; school_id: string; bus_id?: string; bus_number?: string; name: string
 type: TripType; start_point: string; end_point: string; route_qr_code?: string
 stops: Stop[]; is_active: boolean; student_count?: number
 driver_id?: string; driver_name?: string; created_at: string
}

interface Trip {
 id: string; route_id: string; route_name: string; driver_id: string; driver_name: string
 bus_id: string; bus_number: string; trip_type: TripType; status: TripStatus
 started_at?: string; ended_at?: string; student_count: number
}

interface AttendanceRecord {
 id: string; trip_id: string; student_id: string; student_name: string; student_class: string
 stop_id?: string; stop_name?: string; status: AttendanceStatus
 pickup_time?: string; drop_time?: string; route_name?: string; date: string
}

interface Leave {
 id: string; student_id: string; student_name: string; student_class?: string; school_id: string
 from_date: string; to_date: string; reason?: string; status: LeaveStatus
 approved_by?: string; approved_at?: string; created_at: string
}

interface LostFoundItem {
 id: string; school_id: string; bus_id: string; bus_number: string
 driver_id: string; driver_name: string; description: string
 photo_url?: string; image_url?: string; reported_at: string
 status: 'reported' | 'claimed' | 'resolved'; claims: LFClaim[]
}
interface LFClaim {
 id: string; lost_found_id: string; student_id: string; student_name: string
 claim_note?: string; status: 'pending' | 'resolved'; claimed_at?: string
}

interface AppNotification {
 id: string; school_id?: string; user_id?: string; title: string; body: string
 type: 'info'|'warning'|'success'|'error'|'emergency'|'leave'|'attendance'|'message'|'system'
 is_read: boolean; created_at: string; action_url?: string
}
```

```
interface BusTransfer {
 id: string; school_id: string; original_trip_id: string; original_bus_id: string
 original_bus_number: string; new_bus_id: string; new_bus_number: string
 new_driver_id?: string; new_driver_name?: string; authorised_by: string
 transfer_at: string; status: TransferStatus; reason: string; affected_students: number
}

interface GuestTrip {
 id: string; school_id: string; guest_driver_name: string; guest_driver_phone: string
 bus_registration: string; status: GuestTripStatus; approved_by?: string
 started_at?: string; ended_at?: string
 students: Array<{ id: string; name: string; class: string; division: string }>
 created_at: string
}

interface SupportTicket {
 id: string; school_id?: string; school_name?: string
 reporter_id: string; reporter_name: string; reporter_role: UserRole
 type: string; priority: 'low'|'medium'|'high'|'critical'
 status: 'open'|'in_progress'|'resolved'|'escalated'
 description: string; assigned_to?: string; created_at: string; replies: TicketReply[]
}
interface TicketReply {
 id: string; ticket_id: string; user_id: string; user_name: string
 user_role: UserRole; content: string; created_at: string
}
```

---

3. What school-admin manages (and what the mobile app reads/writes into the same tables)

Students

School-admin creates/edits students, assigns them to a route/stop (`pickup_stop_id`/`drop_stop_id`), and records parent contact info (`ParentDetail[]` - see **Gap 1** below, this is *not* linked to a parent's login account). - Mobile touchpoint: parent app would show "my child's" profile, bus, and route - but see Gap 1, this isn't scoped correctly yet.

Buses & Drivers

School-admin manages the fleet (bus records, insurance/fitness expiry tracking) and driver records (license expiry, assigned bus). A `Driver` row has a real `user_id` link to `users` - this is how a driver's mobile login maps to "their" bus/trips. - Mobile touchpoint: driver app's own trips are correctly scoped via this link (`trips.service.isDriverOwnTrip`).

Routes & Stops

School-admin defines routes (pickup/drop), stops with lat/lng and estimated times, and assigns students to stops. - Mobile touchpoint: driver app needs the route's stop list + order to guide the trip; parent app needs the student's assigned stop + ETA.

Trips

Represents one actual run of a route (a bus doing its pickup or drop today). School-admin/driver can update trip status. - Mobile touchpoint: **driver app is the primary actor here** - starts a trip, updates its status (`not_started -> in_progress -> completed`), and this is what the live GPS event ties back to via `trip_id`.

Attendance

School-admin (or driver) marks each student present/absent/leave per trip. There is **no QR-scan flow implemented** - see Gap 4. - Mobile touchpoint: driver app marks attendance manually today (`POST /api/attendance`, `POST /api/attendance/bulk`) - already permitted for the `driver` role.

Leave requests

Any authenticated role can create a leave request for a student; only admin can approve/reject. - Mobile touchpoint: parent app would create leave requests; status changes (approve/reject) stay admin-only, so the parent app should just poll/display status, not attempt to change it.

Lost & Found

Drivers can report a lost item found on their bus; anyone can file/view a claim. - Mobile touchpoint: driver app files reports; parent app can search/claim items.

Bus Transfers

Admin-only - reassigns a live trip to a substitute bus (e.g. breakdown). Creating a transfer immediately repoints `trips.bus_id` and flips both buses' status server-side. - Mobile touchpoint: **read-only** for drivers/parents (`GET` only) - the mobile app should reflect a transfer happening, not initiate one.

Guest Drivers

A one-off substitute driver/trip request. Any authenticated user can create one; only admin can approve/reject; presumably the guest driver themself can mark it `completed`. - Mobile touchpoint: guest driver app creates the request, then - once approved - operates like a normal driver trip (GPS, attendance) for that bus.

Notifications

Pure in-app inbox, always scoped to the requesting user (`req.user.id`). **No push notification is actually sent by the backend** - see Gap 3. - Mobile touchpoint: app can list/mark-read/delete its own notifications; do not expect a push alert to arrive automatically today.

Support Tickets

Any role can open a ticket and reply. Visibility is by `school_id` match OR being the reporter - not filtered to "my tickets only" server-side. - Mobile touchpoint: filter to `reporter_id === currentUser.id` client-side if you only want to show the user's own tickets.

---

4. Real-time: Socket.IO (bus location only)

Same host/port as the REST API. Handshake:

```
io(SOCKET_URL, { auth: { token: jwt } })
```

The server verifies the JWT and joins the socket to room `school:<school_id>` - **there is no per-bus or per-trip room**, so every client in a school receives every bus's location ticks; filter by `bus_id`/`trip_id` on the client.

Driver/guest_driver app emits `bus:location`:

```
{ trip_id, bus_id, latitude, longitude, speed?, current_stop?, status? }
```

Server broadcasts `bus:location` to the whole school room:

```
{ trip_id, bus_id, latitude, longitude, speed: number, current_stop?: string, status: string, recorded_at: strin
```

REST fallback for polling: `GET /api/buses/:id/location` -> `{ location }` (same shape, or `null`).

---

5. Full endpoint reference

Every router requires `Authorization: Bearer <JWT>` except `/api/auth/*` (login/forgot/verify/reset) and `/api/health`.

Prefix	Endpoints	Role gate
`/api/auth`	`POST /login`, `POST /forgot-password`, `POST /verify-otp`, `POST /reset-password`, `GET /me`, `POST /logout`	public / any auth
`/api/students`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin
`/api/drivers`	`GET /expiring-documents`, `GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin
`/api/buses`	`GET /`, `GET /:id`, `GET /:id/location`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin
`/api/routes`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin
`/api/trips`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin, or driver limited to own trip's `status`
`/api/attendance`	`GET /`, `GET /:id`, `POST /`, `POST /bulk`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: admin + driver
`/api/leave`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	any auth (status change gated to admin in controller)
`/api/lost-found`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`, `POST /:id/claims`, `PATCH /:id/claims/:claimId`	GET: any role · report/edit: admin + driver · claims: any auth
`/api/bus-transfers`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`	GET: any role · write: admin only
`/api/guest-trips`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`	create: any auth · approve/reject: admin only
`/api/notifications`	`GET /`, `GET /unread-count`, `POST /`, `PATCH /read-all`, `PATCH /:id/read`, `DELETE /:id`	own-user scoped; create: admin

`/api/messages`	`GET /`, `GET /:id`, `POST /`, `DELETE /:id`	GET: any role · write: admin
`/api/tickets`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `POST /:id/replies`	any auth
`/api/training`	`GET /`, `GET /:id`, `POST /`, `PATCH /:id`, `DELETE /:id`	GET: any role · write: super_admin
`/api/reports`	`GET /attendance-trend`, `GET /fleet-summary`	any auth, tenant-scoped
`/api/reports`	`GET /revenue`, `GET /platform-stats`, `GET /school-growth`	super_admin only
`/api/schools`, `/api/subscriptions`, `/api/plans`, `/api/users`, `/api/audit-logs`	full CRUD	super_admin (+ school_admin read on users/audit-logs) - not relevant to mobile

---

6. Gaps the mobile team needs to plan around

These are real, confirmed gaps in the current backend - not just missing mobile UI. Flagging them now so they're scoped as backend work up front rather than discovered mid-build.

No parent -> student linkage. `ParentDetail` rows have no `user_id` back to `users` - a logged-in parent cannot be resolved to "their" student(s) server-side today. A `parent`-role account can currently list every student in their school via `GET /api/students` (scoped only by `school_id`, same as any other role). **Needs:** a `parent_details.user_id` column (or equivalent) plus a "my children" filter/endpoint before the parent app can safely show only the right student's data.

`fcm_token` has no self-service write path. It's a real column on `users` and is returned in every user-shaped response, but the only endpoint that writes it (`PATCH /api/users/:id`) is restricted to `super_admin`/`school_admin`. **Needs:** a new endpoint such as `PATCH /api/auth/me` or `POST /api/auth/fcm-token` so driver/guest_driver/parent apps can register their own push token.

No push notification sending is implemented anywhere. `fcm_token` is stored but nothing in the backend calls FCM/APNs/webpush - `createNotification` only inserts a DB row. Real-time delivery today is Socket.IO (bus location only) plus polling REST for notifications. **Needs:** actual push-dispatch integration if push alerts are a requirement, not just an in-app inbox.

QR codes exist in the schema but nothing resolves them. `student_qr_code`, `bus.safety_qr_code`, and `route.route_qr_code` are generated at creation and returned in API responses, but there is no "scan -> look up entity" or "scan -> mark attendance" endpoint anywhere. **Needs:** new backend endpoint(s) if a QR-scan attendance flow is wanted (rather than the current manual present/absent/leave selection).

Socket.IO rooms are per-school, not per-bus/trip. Every connected client in a school receives every bus's location ticks; there's no way to subscribe to just one bus server-side. Client-side filtering by `bus_id`/`trip_id` is mandatory.

Support tickets have no "my tickets" filter. Visibility is by school match or being the reporter - filter to `reporter_id === currentUser.id` client-side if the mobile UI should only show a user's own tickets.

---

7. Suggested reading order for the mobile team

1. This document, in full.
2. `backend/src/modules/auth/` - login/token/me.
3. `backend/src/sockets/index.js` + `frontend/src/lib/socket.ts` - live GPS contract.
4. `backend/src/modules/trips/` and `backend/src/modules/attendance/` - the driver app's core loop (start trip -> report location -> mark attendance -> end trip).
5. `backend/src/modules/students/`, `guestTrips/`, `leave/`, `lostFound/`, `notifications/` - parent/guest-driver flows.
6. Section 6 above, before writing any push-notification or "my children" UI.